The Ultimate JavaScript Interview Guide: Concepts & Deep Questions

🟢 Basics & Foundations

1. **Variables & Constants**
    - var, let, const – scope and hoisting
    - **Interview Questions:**
        - What is the difference between var, let, and const?
        - What does temporal dead zone (TDZ) mean in relation to let and const?
        - What is block-level scoping and how is it implemented in JavaScript?
        - Why is var considered function-scoped, and what are the implications?
        - Explain hoisting with examples for var, let, and const.
        - Can a const object have its properties changed?
        - How do you decide when to use let or const?
2. **Data Types**
    - Primitive and Reference types
    - **Interview Questions:**
        - What are the differences between primitive and reference data types?
        - How does JavaScript compare values of different types?
        - Explain the typeof operator and some of its quirks (e.g., typeof null).
        - What is the difference between undefined, null, and NaN?
        - What is the purpose and use-case of Symbol in JavaScript?
        - How does BigInt differ from Number?
3. **Operators**

- Arithmetic, Logical, Comparison, etc.
- **Interview Questions:**
  - What is the difference between == and ===?
  - When would you use the in or instanceof operator?
  - How does the comma operator work and where is it used?
  - What is short-circuit evaluation in logical expressions?
  - Explain optional chaining and how it differs from logical AND (&&).
  - What is the use of the nullish coalescing operator (??) and how does it differ from ||?

4. **Type Conversion**
   - **Interview Questions:**
     - What is the difference between implicit and explicit conversion?
     - How does JavaScript handle type coercion in equality comparisons?
     - How can you convert a number to a string and vice versa?
     - What are edge cases of type coercion (e.g., [] + [], [] + {})?
5. **Control Structures**
   - if, switch, loops
   - **Interview Questions:**
     - How does switch evaluate conditions internally?
     - What's the difference between for...of, for...in, and traditional for loops?
     - When would you use a do...while loop instead of a while loop?
     - How do labeled statements work in nested loops?
6. **Functions**
   - **Interview Questions:**
     - What are first-class and higher-order functions in JavaScript?
     - How do default parameters work and what are their limitations?

- What is the difference between a function declaration and a function expression?
- What is the scope of a function defined inside another function?
- How does the arguments object differ from rest parameters?
- What is function hoisting and how does it differ from variable hoisting?

🟡 Intermediate Concepts

7. **Scope**
    - **Interview Questions:**
        - Explain lexical scoping with an example.
        - How does JavaScript resolve variable references in nested scopes?
        - What is variable shadowing and how can it affect code behavior?
        - How does block scope differ from function scope?
8. **Hoisting**
    - **Interview Questions:**
        - What's hoisted and what's not?
        - Why is accessing a let or const variable before declaration a ReferenceError?
        - How does hoisting affect function expressions vs declarations?
9. **Closures**
    - **Interview Questions:**
        - What are closures and how do they work?
        - How do closures allow for data privacy?
        - Can closures lead to memory leaks? If yes, how?
        - Provide real-world use cases of closures (e.g., setTimeout loop).
10. **IIFE**

- **Interview Questions:**
    - Why were IIFEs commonly used before ES6 modules?
    - What problems do IIFEs solve in global scope?
    - Can you write and explain a named IIFE?

11. **this Keyword**

- **Interview Questions:**
    - How is this determined in different contexts?
    - What is the difference between this in arrow functions vs regular functions?
    - How does this behave in event handlers and class methods?
    - How does strict mode affect the value of this?

12. **Arrow Functions**

- **Interview Questions:**
    - Why can't arrow functions be used as constructors?
    - How do arrow functions capture this from the enclosing lexical scope?
    - Why do arrow functions lack their own arguments object?
    - When should arrow functions be avoided?

13. **Objects**

- **Interview Questions:**
    - How do you merge or clone objects in JavaScript?
    - What is the difference between shallow and deep copy?
    - What do Object.freeze(), Object.seal(), and Object.preventExtensions() do?
    - How do getters and setters work in JavaScript objects?

14. **Arrays**

- **Interview Questions:**
    - What’s the difference between splice() and slice()?
    - How does reduce() work and how can it replace map() and filter()?

- How do you implement deep flattening of a nested array?
- How is immutability maintained while working with arrays?

15. **Destructuring**

- **Interview Questions:**
  - How does nested destructuring work?
  - What happens when destructuring undefined or null values?
  - How can you rename variables during destructuring?

16. **Spread and Rest Operators**

- **Interview Questions:**
  - How does spread work in arrays and objects?
  - How is the rest operator different from the arguments object?
  - Can rest and spread operators be used together?

17. **Template Literals**

- **Interview Questions:**
  - How do template literals allow for multiline strings?
  - What are tagged template literals and how are they useful?
  - Can you use expressions and functions inside template literals?

18. **Truthy/Falsy**

- **Interview Questions:**
  - List all falsy values in JavaScript.
  - How can falsy values affect conditionals and short-circuiting?
  - What's the difference between nullish and falsy values?

19. **Error Handling**

- **Interview Questions:**
  - How does try...catch behave with async/await?
  - What's the difference between a syntax error and a runtime error?

- How can you define and throw custom error types?
- What is the role of finally in error handling?

20. **Date and Time**

- **Interview Questions:**
  - How do you get and format current date/time in JavaScript?
  - How do you compare, subtract, or manipulate date objects?
- What libraries or best practices are recommended for handling dates?

🟠 Advanced Concepts

21. **Prototypes and Prototype Chain**

- **Interview Questions:**
  - What is a prototype in JavaScript?
  - How does JavaScript's prototype chain work?
  - What is the difference between __proto__ and prototype?
  - How can you implement inheritance using prototypes?
  - What is the difference between prototypal and classical inheritance?

22. **Inheritance (ES5 and ES6 Classes)**

- **Interview Questions:**
  - How is inheritance implemented using function constructors?
  - How do ES6 classes simplify inheritance?
  - What does super() do in a constructor?
  - How can you override methods in child classes?

23. **call, apply, and bind**

- **Interview Questions:**
  - How do call(), apply(), and bind() differ?
  - When would you use each of these in real applications?
  - Can you polyfill bind()?

24. **Encapsulation and Closures**

- **Interview Questions:**
    - How do closures help in implementing encapsulation?
    - How would you simulate private variables using closures?
    - What is module pattern in JavaScript?

25. **Currying**

- **Interview Questions:**
    - What is currying and how does it work?
    - How can currying help in function composition?
    - Write a curried version of a function that adds two numbers.

26. **Memoization**

- **Interview Questions:**
    - What is memoization?
    - How does memoization improve performance?
    - Implement a memoization function manually.

27. **Debounce and Throttle**

- **Interview Questions:**
    - What is the difference between debouncing and throttling?
    - What problems do they solve?
    - How would you implement debounce or throttle from scratch?

28. **Event Loop, Call Stack, Microtasks**

- **Interview Questions:**
    - How does the JavaScript event loop work?
    - What is the difference between macro-tasks and micro-tasks?
    - What is the role of Promise.resolve().then() in the event loop?
    - Can you explain how setTimeout(fn, 0) behaves?

29. **Asynchronous JavaScript**

- **Interview Questions:**
    - What is a callback and what are its limitations?
    - What is promise chaining?
    - How does async/await work under the hood?
    - What are common pitfalls when using async/await?

30. **Fetch API / Axios**

- **Interview Questions:**
    - What is the difference between Fetch API and Axios?
    - How does Fetch handle errors differently?
    - How can you cancel a request in Axios?

31. **Modules (CommonJS vs ES Modules)**

- **Interview Questions:**
    - What are ES Modules and how are they different from CommonJS?
    - What is a dynamic import and when would you use it?
    - How do tree shaking and code splitting relate to modules?

32. **Garbage Collection**

- **Interview Questions:**
    - How does JavaScript handle garbage collection?
    - What causes memory leaks?
    - How can closures accidentally retain memory?

🔵 **ES6+ Modern JavaScript**

33. **let & const** – Already covered

34. **Arrow Functions** – Already covered

35. **Default Parameters**

- **Interview Questions:**
    - How do default parameters work in JavaScript?
    - What is the evaluation order for default parameters?

36. **Enhanced Object Literals**

- **Interview Questions:**
    - What are enhanced object literals?
    - How do you define dynamic properties?

37. **Symbols, Sets, Maps**

- **Interview Questions:**
    - What is the use of Symbol()?
    - How does a Map differ from an Object?
    - What are the advantages of Set?

38. **Optional Chaining and Nullish Coalescing** – Already covered

39. **BigInt**

- **Interview Questions:**
    - What problems does BigInt solve?
    - How does BigInt behave in operations with regular numbers?

40. **Dynamic Imports**

- **Interview Questions:**
    - What is dynamic import and how is it used?
    - What are its benefits for lazy loading?

🟣 **Browser Environment APIs**

41. **DOM Manipulation**

- **Interview Questions:**
    - What are different ways to select elements?

- How does event delegation work?
- What are the performance considerations when modifying the DOM?

42. **Event Handling**

- **Interview Questions:**
  - What is event bubbling and capturing?
  - How do you use stopPropagation() and preventDefault()?

43. **Timers (setTimeout / setInterval)**

- **Interview Questions:**
  - How does setTimeout differ from setInterval?
  - What are common timing bugs in JS?

44. **Web Storage**

- **Interview Questions:**
  - What is the difference between LocalStorage, SessionStorage, and Cookies?
  - How would you securely store user data in the browser?

45. **Navigator & History APIs**

- **Interview Questions:**
  - What can you access using the navigator object?
  - How does the History API enable SPA routing?

🔴 **Tooling & Runtime**

46. **Strict Mode**

- **Interview Questions:**
  - What changes does strict mode enforce?
  - Why and when should you use it?

47. **Transpilation and Polyfills**

- **Interview Questions:**

- What is Babel and how does it work?
- What is the difference between transpiling and polyfilling?

48. **Linting and Formatting**

- **Interview Questions:**
  - What does ESLint do and how does it help developers?
  - What are common ESLint rules?

49. **Bundlers (Webpack, Vite)**

- **Interview Questions:**
  - What is a module bundler?
  - How do Webpack and Vite differ?
  - What is tree shaking and code splitting?

50. **Testing (Jest / Mocha)**

- **Interview Questions:**
  - What is unit testing vs integration testing?
  - How does mocking work in Jest?

🧠 **Memory & Performance**

51. **Memory Leaks**

- **Interview Questions:**
  - What causes memory leaks in JavaScript?
  - How do you detect and fix them?

52. **Garbage Collection** – Already covered

53. **Execution Context & Call Stack**

- **Interview Questions:**
  - What happens when JavaScript code runs?
  - Explain the lifecycle of an execution context.

54. **Tail Call Optimization**

- **Interview Questions:**
    - What is tail call optimization and when is it applied?

🧪 Functional Programming

55. **Pure Functions**

- **Interview Questions:**
    - What defines a pure function?
    - Why are pure functions important in functional programming?

56. **Immutability**

- **Interview Questions:**
    - What is immutability and why is it beneficial?
    - How can you ensure immutability in JS code?

57. **Higher-Order Functions**

- **Interview Questions:**
    - What is a higher-order function?
    - Give examples using map, filter, reduce.

58. **Function Composition**

- **Interview Questions:**
    - What is function composition and why use it?
    - How can you implement a compose function?

🧩 Expert-Level JavaScript Concepts

59. **Advanced Event Loop & Concurrency**

- **Interview Questions:**
    - What is the difference between queueMicrotask() and Promise.then()?
    - How does JavaScript prioritize rendering vs executing JavaScript?

- Explain message queues and job queues in the event loop.
- How do browsers optimize or batch DOM rendering and reflows?

60. **Performance Optimization Techniques**

- **Interview Questions:**
  - How can you optimize rendering performance in large DOM trees?
  - What are techniques to minimize layout thrashing?
  - How would you improve JS performance for real-time search or filtering?
  - What is lazy loading and how is it implemented with vanilla JS?

61. **Security in JavaScript**

- **Interview Questions:**
  - What is XSS and how can you prevent it in your code?
  - What is CSRF and what defense mechanisms exist for it?
  - How does a Content Security Policy (CSP) mitigate JavaScript attacks?
  - Why should eval() and Function() be avoided?

62. **Meta Programming with Proxy & Reflect**

- **Interview Questions:**
  - What is a Proxy and how would you use it?
  - What is the role of the Reflect object?
  - Give an example of logging or validation using Proxy.

63. **Memory Profiling & DevTools**

- **Interview Questions:**
  - How can you detect memory leaks using Chrome DevTools?
  - What are detached DOM nodes and how do they cause memory issues?
  - How can timeline and heap snapshot help analyze performance?

64. **Web Workers & Multithreading**

- **Interview Questions:**

- What are Web Workers and when should they be used?
- How do you communicate between the main thread and a worker?
- What are limitations of Web Workers?

65. **Advanced Symbol Use-Cases**

- **Interview Questions:**
  - How can Symbols be used to create private-like object properties?
  - What are well-known Symbols such as Symbol.iterator and Symbol.toPrimitive?
  - How can you customize object behavior using Symbol methods?

66. **Advanced Module Design**

- **Interview Questions:**
  - What is an Immediately Loaded Module (ILM)?
  - How do asynchronous and synchronous module loading differ?
  - What are the downsides of circular dependencies in modules?
